WORKSTREAM

Functionality Reimagination Proposal

Inventory Management Application

A forward-looking proposal to transform static AS/400 inventory management into an intelligent, real-time, data-driven platform — unlocking capabilities that were impossible within the constraints of legacy RPG green-screen architecture.

Application: TRTCT004 — Inventory Management System
Current Stack: Spring Boot 3.3 · React 18 · PostgreSQL 16
Legacy Origin: IBM AS/400 RPG IV · DB2/400 · DSPF 5250
Date: April 28, 2026
Classification: Internal — Strategic Planning

PROPOSAL · VERSION 1.1 · SCOPE REVISED

Table of Contents

1 Executive Summary

2 Current State Assessment

3 Reimagination Vision & Principles

4 Proposed Functionality Reimaginations

4.1 Real-Time Inventory Intelligence Dashboard

4.2 Predictive Reorder & Demand Forecasting

4.3 Barcode / QR Code Scanning Integration

4.4 Multi-Warehouse Transfer Management

4.5 Complete Audit Trail & Transaction History

4.6 ABC Inventory Classification & Analytics

4.7 Automated Alerts & Notification Engine

4.8 Bulk Operations & Import/Export

4.9 Supplier Lead Time & Goods Receipt (Scoped)

4.10 Mobile-First Warehouse Floor Application

4.11 Cycle Counting & Physical Inventory

4.12 Lot Tracking, Expiry & Serial Number Management

4.13 Inventory Valuation & Costing Methods

Cross-Cutting Infrastructure Prerequisites

4.14 Role-Based Access Control (Infrastructure)

4.15 API Documentation & Inventory Webhooks (Infrastructure)

5 Implementation Roadmap

6 Impact Analysis & Priority Matrix

7 Technical Architecture Evolution

8 Risk Assessment & Mitigation

9 Success Metrics & KPIs

10 Summary & Recommendation

1. Executive Summary

The TRTCT004 Inventory Management System has been successfully migrated from IBM AS/400 RPG to a modern Spring Boot + React stack. While this migration preserves all 70+ legacy business rules, it only scratches the surface of what a modern platform can deliver. This proposal outlines **12 inventory-domain functionality reimaginations** plus **2 cross-cutting infrastructure prerequisites** that transform inventory management from a transactional record-keeping system into an **intelligent, predictive, and connected** platform — all strictly within the scope of inventory management.

12	2	3	8
INVENTORY REIMAGINATIONS	INFRASTRUCTURE PREREQUISITES	IMPLEMENTATION PHASES	NEW SCREENS

> **Core Thesis:** The legacy system asked "*What do we have?*" — the reimagined system answers "*What should we have, when, where, and why?*" Moving from reactive record-keeping to proactive inventory intelligence.

DIMENSION	CURRENT STATE (MIGRATED)	REIMAGINED STATE (PROPOSED)
Inventory Visibility	Manual browse with basic category/name filters	Real-time dashboard with KPIs, trends, and heatmaps
Reorder Decision	Static reorder point comparison (ALG-002)	ML-driven demand forecasting with seasonal adjustment
Data Entry	Manual form-based item creation	Barcode/QR scan, bulk import, supplier auto-feed
Warehouse Ops	Single-warehouse view per item	Multi-warehouse transfer, mobile floor app
Audit & Compliance	Transaction table exists but unwritten (INVTRN)	Full audit trail, cycle counting, lot tracking
Integration	Standalone application, static reference adapters	Database-backed reference adapters, API documentation, inventory event webhooks

2. Current State Assessment

2.1 What Has Been Achieved

The migration from AS/400 to Spring Boot + React has delivered a functionally equivalent modern application:

ASPECT	LEGACY (AS/400)	CURRENT (MIGRATED)
Architecture	Monolithic RPG programs (PGM-001–004)	Hexagonal architecture with ports & adapters
UI	6 DSPF green screens (5250 terminal)	6 React screens with glassmorphic dark theme
Data	DB2/400 physical files (PF/LF)	PostgreSQL 16 with JPA + Flyway migrations
State Management	Implicit in RPG indicators	Explicit state machine (STM-001) with EnumMap
Validation	RPG IF/ELSE chains	Dual-layer: Zod (frontend) + Spring validator (backend)
Business Rules	70+ rules embedded in RPG source	70+ rules preserved and traceable (BR-001 through BR-070)
Testing	Manual QA only	120 automated unit tests (JUnit 5 + Mockito + AssertJ)

2.2 Current Functional Boundaries

The migrated application faithfully replicates — but does not extend — the following functional scope:

SCREEN	FUNCTION	CAPABILITIES	LIMITATIONS
SCR-001	Main Menu	3-option navigation (Browse, Add, Reorder)	No dashboard, no at-a-glance metrics
SCR-002	Item List	Filter by category/name, Edit/View/Delete actions	200-row cap, no sorting, no export, no pagination
SCR-003	Item Detail	Add/Edit/Display modes, validation, margin calc	No history, no image, no related items, no attachments
SCR-004	Delete Confirm	Soft-delete with allocation check	No bulk delete, no undo capability
SCR-005	Exit Confirm	Session termination guard	Legacy pattern, unnecessary in web context

SCR-006	Reorder Report	Shortage calc, replenishment value, filters	Static thresholds only, no forecasting, no auto-order

2.3 Identified Gaps & Unrealized Potential

GAP-1 Dormant Transaction Table

The INVTRN (inventory transaction log, DB-005) table was created in V1 migration with full schema — but no service, repository, or controller writes to it. This eliminates all audit capability.

GAP-2 Static Reference Adapters

Category, Supplier, and Warehouse adapters use hardcoded static maps instead of querying the database tables (CTGMST, SUPLMST, WHSMST) that were seeded in V5 migration.

GAP-3 No Inter-Warehouse Visibility

Each item is assigned to a single warehouse. There is no mechanism for warehouse-to-warehouse transfers, consolidated stock views, or optimal sourcing logic.

GAP-4 Report Print Stub

ReportPrintAdapterProxy.spoolReport() is a stub. Legacy INVRPTR (PGM-004) printed physical spool files — this capability was not re-implemented, leaving reporting incomplete.

3. Reimagination Vision & Principles

3.1 Vision Statement

> Transform WorkStream Inventory Management from a **transactional record system** into an **intelligent inventory platform** that predicts demand, automates replenishment, enables real-time warehouse operations, and provides actionable business intelligence — all while preserving the 70+ battle-tested business rules from the legacy system.

3.2 Guiding Principles

#	PRINCIPLE	DESCRIPTION
P1	**Intelligence Over Entry**	Automate decisions that were previously manual. The system should recommend, not just record.
P2	**Real-Time Over Batch**	Replace legacy batch processing patterns with event-driven, real-time data flows.
P3	**Predictive Over Reactive**	Use historical data and algorithms to anticipate needs instead of responding to shortages after they occur.
P4	**Connected Over Isolated**	Break silos between inventory, suppliers, warehouses, and payroll systems through APIs and integrations.
P5	**Mobile Over Desk-Bound**	Bring inventory management to the warehouse floor with mobile-first experiences.
P6	**Auditable Over Opaque**	Every inventory movement must be traceable, compliant, and explainable.
P7	**Preserve & Extend**	All 70+ existing business rules (BR-001 through BR-070) remain intact. New capabilities layer on top, never replace validated logic.

4. Proposed Functionality Reimaginations

4.1 Real-Time Inventory Intelligence Dashboard

FR-001 Replace static menu (SCR-001) with a live intelligence dashboard

Legacy Context: SCR-001 (INVMNU display file) was a simple 3-option text menu — Browse, Add Item, Reorder Report. It provided zero visibility into inventory health on entry.

Reimagined Capability:

FEATURE	DESCRIPTION	TYPE
Inventory Health Score	Composite KPI (0–100) based on stock levels, aging, allocation ratios, and margin distribution	NEW
Stock Level Heatmap	Visual heatmap across warehouses showing overstocked (blue), optimal (green), low (yellow), critical (red) items	NEW
Trending Items Widget	Top 10 fastest-moving and slowest-moving items over configurable timeframes (7d, 30d, 90d)	NEW
Reorder Urgency Ticker	Live count of items below reorder point, segmented by urgency (critical/warning/watch)	ENHANCED
Financial Summary	Total inventory value, total margin, cost breakdown by category — derived from existing ALG-001	ENHANCED
Recent Activity Feed	Last 20 transactions — additions, edits, deletions, reorder events — pulled from INVTRN (once FR-005 activates it)	NEW

Technical Approach: New `DashboardService` aggregating data from existing `InventoryItemRepository` queries plus new aggregate SQL queries. React component using MUI charts (recharts or nivo) with WebSocket push for real-time updates.

4.2 Predictive Reorder & Demand Forecasting

FR-002 Evolve static reorder points into ML-driven demand prediction

Legacy Context: The reorder report (SCR-006) uses a simple threshold comparison: `quantityOnHand ≤ reorderPoint` (BR-062). Reorder points and quantities are manually maintained per item — they never adapt to actual demand patterns.

Reimagined Capability:

FEATURE	DESCRIPTION	TYPE
Consumption Velocity Tracking	Calculate daily/weekly burn rate per item from transaction history. Replace static reorderPoint with dynamic safety stock levels.	REIMAGINE
Seasonal Demand Curves	Identify seasonal patterns (quarterly, monthly) and adjust reorder timing proactively. Items ordered 2 weeks before predicted demand spike.	NEW
Lead Time Awareness	Factor supplier lead times into reorder calculations. Trigger reorder when `daysOfStockRemaining < supplierLeadTimeDays`.	NEW
Economic Order Quantity (EOQ)	Calculate optimal order quantity using EOQ formula: `√(2DS/H)` where D=annual demand, S=order cost, H=holding cost per unit.	REIMAGINE
Auto-Reorder Suggestions	System generates purchase order drafts when predicted stockout date falls within lead time window. Requires human approval to execute.	NEW
Forecast Accuracy Tracking	Compare predicted vs actual demand over time. Display forecast accuracy percentage to build system trust.	NEW

Algorithm Evolution: ALG-002 (shortage = max(0, reorderPoint − qtyOnHand)) remains as the fallback. The new predictive layer sits above it: `predictedShortage = max(0, forecastedDemand(leadTimeDays) − qtyOnHand + safetyStock)`. If forecasting data is insufficient (<90 days of history), the system gracefully degrades to the original ALG-002 logic.

4.3 Barcode / QR Code Scanning Integration

FR-003 Enable scan-to-manage inventory operations

Legacy Context: All item identification on AS/400 was manual keyboard entry of 8-character item IDs (F-003-01, field IMITID). Warehouse staff had to memorize or look up IDs from printed reports.

Reimagined Capability:

FEATURE	DESCRIPTION
Camera-Based Scanning	Use device camera (mobile or webcam) to scan barcodes/QR codes via `quagga2` or `html5-qrcode` library. Instant item lookup without typing.
Barcode Generation	Auto-generate Code 128 barcodes and QR codes for each item. Printable labels with item ID, name, warehouse, and category.
Scan-to-Count	Scan an item → system shows current QOH → user enters counted quantity → variance recorded. Supports cycle counting (FR-011).
Scan-to-Receive	Scan incoming shipment items to auto-increment quantityOnHand and log receipt transaction in INVTRN.
Batch Scan Mode	Continuous scanning mode: scan multiple items in sequence, building a batch list for bulk operations (FR-008).

4.4 Multi-Warehouse Transfer Management

FR-004 Enable cross-warehouse stock transfers and consolidated visibility

Legacy Context: Each inventory item is assigned to a single warehouse (field IMWHSE, DB-001). The legacy system had no concept of transferring stock between warehouses (WH01–WH04). Overstocking in one warehouse and stockouts in another were invisible.

Reimagined Capability:

FEATURE	DESCRIPTION
Consolidated Stock View	New screen showing total quantity across all warehouses per item. Drill-down to per-warehouse breakdown with visual bar chart.
Transfer Request Workflow	Create transfer requests: source warehouse, destination warehouse, item, quantity. Two-step approval: initiate → confirm receipt.
Transfer-in-Transit Status	New state: TRANSFER_IN_TRANSIT added to item lifecycle. Quantity deducted from source, held in transit, added to destination on confirmation.

Optimal Sourcing Suggestion	When fulfilling demand, system suggests which warehouse to source from based on stock levels, proximity, and cost.
Transfer History Log	All transfers recorded in INVTRN with type='TR'. Full audit trail with timestamps, users, and quantities.

Database Changes: New `inventory.whtransfer` table for transfer workflow. New INVTRN transaction type 'TR' (transfer). New composite index on (itemId, warehouseCode) for consolidated queries.

4.5 Complete Audit Trail & Transaction History

FR-005 Activate the dormant INVTRN table with full event sourcing

Legacy Context: The INVTRN table (DB-005) was defined in V1 migration with fields for transaction type, quantity, date, user, and reference — but **no code writes to it** (SME-004). Every inventory change is currently a silent overwrite with no history.

Reimagined Capability:

TRANSACTION TYPE	CODE	TRIGGER	RECORDED DATA
Item Created	CR	saveItem() in Add mode	Initial quantities, cost, price, user, timestamp
Item Updated	UP	saveItem() in Edit mode	Changed fields (before/after), user, timestamp
Quantity Adjusted	QA	Manual quantity change	Old qty, new qty, adjustment reason, user
Status Changed	SC	STM-001 transition	From status, to status, transition ID, user
Transfer Out	TO	Warehouse transfer (FR-004)	Source warehouse, quantity, destination, user
Transfer In	TI	Transfer receipt	Receiving warehouse, quantity, source, user
Stock Received	SR	Purchase order receipt	Supplier, PO reference, quantity, cost, user
Cycle Count	CC	Cycle count adjustment (FR-011)	Expected qty, counted qty, variance, user

New Screen — SCR-007: Transaction History Viewer

- Filterable by item ID, transaction type, date range, user, warehouse
- Timeline visualization showing quantity changes over time
- Export to CSV/PDF for compliance and audit reporting
- Linked from item detail screen (SCR-003) via "View History" button

Implementation: New `InventoryTransactionService` with `@TransactionalEventListener` pattern — transaction events are published by existing services and captured asynchronously, ensuring zero performance impact on the primary write path.

4.6 ABC Inventory Classification & Analytics

FR-006 Auto-classify inventory by value contribution using Pareto analysis

Legacy Context: All inventory items are treated equally in the legacy system. A $0.50 packing material receives the same reorder attention as a $5,000 electronic component. There is no prioritization mechanism.

Reimagined Capability:

CLASS	RULE	TYPICAL %	MANAGEMENT POLICY
A — Critical	Top items contributing to 80% of total inventory value	~20% of items	Tight control, frequent cycle counts, low safety stock, demand forecasting enabled
B — Important	Next items contributing to 15% of total value	~30% of items	Moderate control, quarterly cycle counts, standard reorder process
C — Standard	Remaining items contributing to 5% of value	~50% of items	Simplified control, annual counts, higher safety stock to reduce order frequency

Algorithm — ALG-004: ABC Classification

```
1. For each ACTIVE item: annualValue = quantityOnHand × unitCost
2. Sort all items by annualValue descending
3. Calculate cumulative percentage of total value
4. Assign class:
   - A: cumulative% ≤ 80%
   - B: 80% < cumulative% ≤ 95%
   - C: cumulative% > 95%
5. Store classification on entity (new field: abcClass CHAR(1))
6. Recalculate nightly via scheduled job
```

New Screen Elements: ABC class badge on list screen (SCR-002), ABC distribution donut chart on dashboard (FR-001), classification override capability for manual adjustment.

4.7 Automated Alerts & Notification Engine

FR-007 Push-based alerting for critical inventory events

Legacy Context: On the AS/400, users discovered problems only when they manually ran reports or navigated to a specific screen. There was no proactive alerting — if a critical item hit zero stock on a Friday, nobody knew until Monday.

Reimagined Capability:

ALERT TYPE	TRIGGER CONDITION	CHANNEL
Critical Stock Alert	quantityOnHand = 0 for any ACTIVE item	Email, In-App, SMS
Reorder Warning	quantityOnHand ≤ reorderPoint (BR-062)	In-App, Email digest
Predicted Stockout	Forecasted stockout within lead time window (FR-002)	In-App, Email
High-Value Variance	Cycle count variance >5% on Class A items (FR-006)	Email, Manager alert
Price Below Cost	unitPrice < unitCost (BR-045 violation attempt)	In-App warning
Bulk Status Change	>10 items deactivated in a single day	Email to admin
Supplier Lead Time Breach	Expected delivery date exceeded	In-App, Email

Architecture: Event-driven notification engine using Spring Events. Configurable alert preferences per user (notification settings screen). In-app notification bell on AppLayout.tsx header with unread count badge. Email integration via Spring Mail with HTML templates.

4.8 Bulk Operations & Import/Export

FR-008 **Enable mass data operations that were impossible on 5250 terminals**

Legacy Context: Every operation on the AS/400 was one-item-at-a-time via EXFMT display file I/O. Updating 500 items meant 500 manual screen interactions. Importing data required DFU or custom CL programs with strict IBM formatting.

Reimagined Capability:

OPERATION	DESCRIPTION
CSV/Excel Import	Upload spreadsheet of items for bulk creation. Column mapping wizard. Validation preview showing errors before commit. Supports Add and Update modes.
CSV/Excel/PDF Export	Export current list view (with active filters) to downloadable file. Replaces legacy INVRPTR spool file printing. Includes all visible columns plus optional extended fields.
Bulk Status Update	Select multiple items from list → apply status change to all. Respects STM-001 transition rules per item; reports individual failures without rolling back successes.
Bulk Price Adjustment	Apply percentage or flat-rate price change across selected items or category. Preview impact (total value change, affected margins) before confirming.
Bulk Category Reassignment	Move items between categories. Validates target category exists (BR-039) for each item.

Template Download	Download pre-formatted CSV template with headers matching InventoryItemRequestDto fields. Includes sample data and field descriptions.

4.9 Supplier Lead Time & Goods Receipt

FR-009 Enrich supplier reference data and enable inventory receipt tracking

> **Scope Note:** This reimagination has been intentionally scoped to inventory-relevant supplier capabilities only. Purchase Order lifecycle management, Supplier Portal, and Supplier Scorecards belong in a future **Procurement Module** and are explicitly **out of scope** for the Inventory Management application.

Legacy Context: Suppliers exist only as 6-character codes (SUP001–SUP005) in the SUPLMST table. The SupplierReferenceAdapter currently uses a hardcoded static map instead of querying the database. Inventory has no awareness of supplier lead times, making predictive reorder (FR-002) impossible.

In-Scope Reimagined Capability:

FEATURE	DESCRIPTION	TYPE
Database-Backed Supplier Adapter	Replace static SupplierReferenceAdapter with JPA-backed adapter querying SUPLMST table. All 3 reference adapters (Category, Supplier, Warehouse) become database-driven, closing GAP-2.	ENHANCED
Supplier Lead Time Field	Add `lead_time_days` (INTEGER) to SUPLMST. Used by FR-002 (Predictive Reorder) to calculate when to trigger reorder: `daysOfStockRemaining < supplierLeadTimeDays`.	NEW
Goods Receipt Entry	Simple receipt screen: select item, enter received quantity, confirm supplier. System auto-updates quantityOnHand and logs SR (Stock Received) transaction in INVTRN. No PO reference required.	NEW
Receipt Discrepancy Flag	If received quantity differs from reorderQuantity on item, flag for review. Informational only — does not block receipt.	NEW

Explicitly Out of Scope (Deferred to Procurement Module):

CAPABILITY	REASON FOR EXCLUSION	BELONGS IN
Purchase Order CRUD & Lifecycle	PO management (Draft → Submitted → Shipped → Received → Closed) is procurement domain logic, not inventory	Procurement Module
Supplier Portal	External-facing supplier self-service portal is a Supplier Relationship Management (SRM) concern	SRM Module
Supplier Scorecards	On-time delivery %, quality metrics, and supplier ranking belong in procurement analytics	Procurement Module

Automated PO Generation	Auto-creating POs from reorder suggestions requires procurement approval workflow	Procurement Module

Database: Extend SUPLMST with `lead_time_days` INTEGER DEFAULT 7. No new tables required — goods receipts are logged as SR transactions in the existing INVTRN table (activated by FR-005).

4.10 Mobile-First Warehouse Floor Application

FR-010 **Progressive Web App (PWA) optimized for warehouse operations**

Legacy Context: AS/400 access required dedicated 5250 terminals — large, desk-mounted devices. Warehouse staff walked to a terminal station, performed lookups, then walked back to shelves. Disconnected physical workflow created delays and errors.

Reimagined Capability:

FEATURE	DESCRIPTION
PWA with Offline Support	Service worker caches critical data for offline viewing. Queues updates when offline and syncs when connection restores. Works on any mobile browser.
Quick-Look Item Card	Scan barcode (FR-003) → instantly see: item name, QOH, location, reorder status, last movement date. One-tap to adjust quantity.
Warehouse Walk-List	Optimized pick/count list ordered by physical location (aisle → shelf → bin). Minimizes walking distance for cycle counts and order picking.
Touch-Optimized Quantity Entry	Large +/− buttons and numeric keypad for gloved hands. Confirmation haptic feedback.
Photo Attachment	Take photos of damaged goods, shelf labels, or shipment documents. Attached to item record or transaction for evidence.

Technical Approach: Extend existing React app with responsive breakpoints and PWA manifest. Use `workbox` for service worker. Camera API for barcode scanning. IndexedDB for offline queue. Same Spring Boot REST backend — no server changes required.

4.11 Cycle Counting & Physical Inventory

FR-011 **Replace disruptive full physical inventory with continuous cycle counting**

Legacy Context: The legacy system had no physical inventory or count functionality. Discrepancies between system quantities and actual shelf quantities accumulated silently. The only remedy was offline spreadsheet-based physical counts — a disruptive, labor-intensive annual event.

Reimagined Capability:

FEATURE	DESCRIPTION
Count Schedule Generator	Auto-generate count schedules based on ABC classification (FR-006): Class A = monthly, B = quarterly, C = annually. Distributes counting workload evenly across calendar days.
Count Assignment	Assign specific items to specific users for counting. Track completion status: PENDING → IN_PROGRESS → COMPLETED → REVIEWED.
Count Entry & Variance	User enters physical count → system calculates variance (system QOH vs counted). Auto-flag variances exceeding configurable threshold (default: 5% for A, 10% for B, 15% for C).
Variance Investigation	For flagged variances: require supervisor review and reason code (Damaged, Misplaced, Theft, Receiving Error, Shipping Error, Other). Logged in INVTRN.
Auto-Adjustment	After supervisor approval, system adjusts quantityOnHand to match counted value. Generates QA transaction in INVTRN with full audit trail.
Count Accuracy Dashboard	Track inventory accuracy over time: % of items counted, average variance, accuracy by warehouse/category/ABC class.

4.12 Lot Tracking, Expiry & Serial Number Management

FR-012 Enable traceability from receipt through consumption

Legacy Context: The AS/400 system tracks only aggregate quantities per item (IMQTOH). It has no concept of lots, batches, expiration dates, or serial numbers. If a recall is issued for a specific manufacturing batch, there is no way to identify affected stock.

Reimagined Capability:

FEATURE	DESCRIPTION
Lot/Batch Numbers	Associate received items with lot numbers from suppliers. Track quantity per lot within an item. Enable FIFO (First In, First Out) consumption.
Expiry Date Management	Record expiry date per lot. Dashboard widget showing items expiring within 30/60/90 days. Auto-alert for items nearing expiry (FR-007).
Serial Number Tracking	For high-value items (Class A): assign unique serial numbers. Full lifecycle tracing from receipt → location → issuance → return.
Recall Management	Enter lot number or date range → system identifies all affected stock locations, quantities, and status. Supports quarantine status on affected lots.
FEFO Picking Logic	First Expiry, First Out: when allocating stock, system prioritizes lots with nearest expiry date to minimize waste.

Database: New table `inventory.lotmst` (lot master): lot_id, item_id, lot_number, supplier_lot_ref, received_date, expiry_date, quantity, status. New table `inventory.serialmst`: serial_id, item_id, lot_id, serial_number, current_location, status.

4.13 Inventory Valuation & Costing Methods

FR-013 Support multiple costing methodologies for financial reporting

Legacy Context: The current system stores a single `unitCost` per item — effectively a standard/replacement cost. There is no historical cost tracking, no weighted average calculation, and no FIFO/LIFO costing. ALG-001 calculates margin based solely on the current unit cost.

Reimagined Capability:

COSTING METHOD	DESCRIPTION
Standard Cost	Current approach (preserved): fixed unitCost manually maintained. Variance = actual purchase price − standard cost.

Weighted Average Cost	Recalculate cost on each receipt: `newAvgCost = (existingQty × currentCost + receivedQty × purchasePrice) / (existingQty + receivedQty)`
FIFO Costing	Track cost per lot (FR-012). COGS uses oldest lot cost first. Requires lot-level quantity tracking.
LIFO Costing	COGS uses most recent lot cost first. For tax optimization in specific jurisdictions.

New Capabilities:

- **Inventory Valuation Report:** Total value calculated under selected costing method. Compares valuations across methods side-by-side.
- **Margin Re-Calculation:** ALG-001 enhanced to use selected costing method instead of static unitCost. Margin becomes dynamic based on actual historical costs.
- **Cost Variance Analysis:** Dashboard showing purchase price variance (PPV) trends over time, by supplier, by category.

Cross-Cutting Infrastructure Prerequisites

> **Scope Classification:** The following two items (FR-014, FR-015) are **not inventory-domain reimaginations** — they are **cross-cutting infrastructure prerequisites** that any production application requires regardless of business domain. They are included in this proposal because the inventory application depends on them, but they are domain-agnostic capabilities that would apply equally to the Payroll module or any other application in the TRTCT004 ecosystem.

4.14 Role-Based Access Control INFRASTRUCTURE

FR-014 Secure the application with authentication and granular permissions

Legacy Context: The AS/400 relied on OS-level security profiles (USRPRF objects) and library-level authority. The migrated application currently has **no authentication or authorization** — every endpoint is publicly accessible.

Why It's Infrastructure, Not Inventory: Every web application needs authentication and authorization. These roles, JWT tokens, and SSO integration are not specific to inventory management — the same security layer would be needed for Payroll, Procurement, or any other module.

Required Capability:

ROLE	PERMISSIONS	USE CASE
Viewer	Read-only access to all screens. Cannot add, edit, or delete items.	Auditors, management
Operator	Create/edit items, submit cycle counts, scan operations. Cannot delete or change status.	Warehouse staff
Manager	All Operator permissions + soft delete, approve cycle count variances, run reports.	Warehouse managers
Admin	Full access including reference data management, user management, system configuration, bulk operations.	IT / system admin

Implementation:

- Spring Security with JWT tokens (stateless auth, replacing legacy stateful session model)
- Warehouse-scoped data isolation: Operators see only their assigned warehouse's items
- Audit log of all login/logout events and permission-denied attempts
- PII masking on supplier contact fields (SUPLMST) based on role (addresses GAP in DB-003 PII annotations)

- OAuth2/OIDC integration for enterprise SSO

4.15 API Documentation & Inventory Webhooks INFRASTRUCTURE

FR-015 **Document inventory APIs and enable event-driven integration**

Legacy Context: The AS/400 system communicated with other programs only through data queues, data areas, and CALL/PARM — tightly coupled, synchronous, and IBM-only. The migrated system has REST endpoints but no API documentation or event publishing.

Why It's Infrastructure, Not Inventory: API documentation (OpenAPI/Swagger) and webhook infrastructure are generic platform capabilities. The webhook engine, API key management, rate limiting, and event bus architecture would serve any module — not just inventory.

Inventory-Relevant Scope:

FEATURE	DESCRIPTION
OpenAPI 3.0 Specification	Auto-generated API documentation via springdoc-openapi for all inventory endpoints. Interactive Swagger UI at /api-docs.
Inventory Event Webhooks	Publish inventory-specific events: item.created, item.updated, item.statusChanged, stock.critical, reorder.triggered. External systems subscribe per event type.
Rate Limiting & API Keys	Protect public API endpoints with rate limiting. API key management for external consumers.

Explicitly Out of Scope (Platform-Level Concerns):

CAPABILITY	REASON	BELONGS IN
Kafka/Event Bus Infrastructure	Message broker setup is a platform ops decision, not an inventory feature	Platform/DevOps
ERP/Accounting Feeds	Cross-system data pipelines span multiple domains	Integration Layer
Payroll System Integration	Cross-module integration belongs at the platform orchestration level	Platform/Integration

5. Implementation Roadmap

5.1 Phase Overview

PHASE	THEME	DURATION	REIMAGINATIONS INCLUDED
PHASE 1	**Foundation & Visibility**	8–10 weeks	FR-005 (Audit Trail), FR-001 (Dashboard), FR-008 (Bulk Ops), FR-014 (RBAC) INFRA, FR-015 (API Docs) INFRA
PHASE 2	**Intelligence & Operations**	10–12 weeks	FR-002 (Predictive Reorder), FR-006 (ABC Class), FR-007 (Alerts), FR-009 (Supplier Lead Time & Receipt), FR-011 (Cycle Count)
PHASE 3	**Advanced & Mobile**	10–14 weeks	FR-003 (Barcode), FR-004 (Multi-Warehouse), FR-010 (Mobile PWA), FR-012 (Lot Tracking), FR-013 (Costing)

5.2 Phase 1 — Foundation & Visibility (Weeks 1–10)

Build the data infrastructure and visibility that all subsequent phases depend on.

WEEK	FR	MILESTONE
1–2	FR-005	Implement InventoryTransactionService. Wire event listeners to existing saveItem(), softDeleteItem(), and state machine transitions. All CRUD operations now write to INVTRN. Backfill test data.
2–3	FR-014 INFRA	Integrate Spring Security with JWT. Define 4 roles (Viewer, Operator, Manager, Admin). Secure all existing endpoints. Add login screen to React app.
3–5	FR-001	Build dashboard screen (SCR-007). Aggregate queries for KPIs. Stock level visualization. Recent activity feed (powered by FR-005 INVTRN data). Replace SCR-001 menu with dashboard + navigation sidebar.
5–7	FR-008	CSV import with validation preview. Export current list to CSV/Excel/PDF. Bulk status update from list selection. Replaces ReportPrintAdapterProxy stub (GAP-4).
7–8	FR-015 INFRA	Add springdoc-openapi to pom.xml. Annotate all controllers with @Operation. Publish Swagger UI at /api-docs. Configure inventory event webhooks.
9–10	—	Integration testing, UAT, Phase 1 deployment. Verify all 70+ existing BRs still pass. Run full 120-test suite + new tests.

5.3 Phase 2 — Intelligence & Operations (Weeks 11–22)

Layer analytical intelligence and operational workflows on top of the data foundation.

WEEK	FR	MILESTONE
11–12	FR-006	Implement ALG-004 ABC classification. Add abcClass field to InventoryItemEntity. Nightly scheduled recalculation. ABC badges on list and detail screens.
13–15	FR-002	Build consumption velocity tracking from INVTRN data. Implement EOQ calculator. Predictive stockout date estimation. Enhanced reorder report (SCR-006) with forecasting columns.
15–17	FR-007	Event-driven notification engine. 7 alert types. In-app notification center. Email integration. User notification preferences screen.
17–19	FR-009	Replace static supplier adapter with database-backed JPA adapter. Add lead_time_days to SUPLMST. Build goods receipt entry screen. Wire SR transactions to INVTRN.
19–21	FR-011	Cycle count schedule generator (driven by FR-006 ABC class). Count assignment, entry, and variance workflow. Auto-adjustment with supervisor approval. Count accuracy dashboard.
21–22	—	Integration testing, UAT, Phase 2 deployment.

5.4 Phase 3 — Advanced & Mobile (Weeks 23–36)

Deliver advanced capabilities and bring the application to the warehouse floor.

WEEK	FR	MILESTONE
23–25	FR-003	Integrate html5-qrcode library. Camera-based scanning. Barcode generation and label printing. Scan-to-count and scan-to-receive flows.
25–28	FR-004	Multi-warehouse transfer workflow. Consolidated stock view screen. Transfer-in-transit status. Optimal sourcing algorithm. Transfer audit trail.
28–31	FR-012	Lot tracking data model. Expiry date management. FEFO picking logic. Recall management screen. Serial number tracking for Class A items.
31–33	FR-013	Weighted average cost calculation. FIFO/LIFO costing (depends on FR-012 lots). Inventory valuation report. Multi-method margin comparison.
33–35	FR-010	PWA manifest and service worker. Offline support via workbox. Touch-optimized mobile layouts. Walk-list generation. Photo attachments.
35–36	—	End-to-end testing, performance tuning, Phase 3 deployment.

6. Impact Analysis & Priority Matrix

6.1 Business Value vs Implementation Effort

FR	REIMAGINATION	BUSINESS VALUE	EFFORT	PHASE	RATIONALE
FR-005	Audit Trail	HIGH	LOW	P1	Table exists; just need writers. Unlocks FR-001, FR-002, FR-011 data needs.
FR-014	RBAC INFRA	HIGH	MED	P1	Security is non-negotiable for production. Blocks all external-facing features. Cross-cutting infrastructure.
FR-001	Dashboard	HIGH	MED	P1	Highest visibility improvement. Transforms user experience from day one.
FR-008	Bulk Operations	HIGH	MED	P1	Massive time savings. Single biggest complaint about legacy one-at-a-time UI.
FR-002	Predictive Reorder	HIGH	HIGH	P2	Highest ROI potential (reduced stockouts + reduced overstock). Needs INVTRN data from P1.
FR-006	ABC Classification	MED	LOW	P2	Simple algorithm, high organizational value. Required input for FR-011 counting schedules.
FR-007	Alerts	HIGH	MED	P2	Proactive vs reactive. Prevents stockout losses.
FR-009	Supplier Lead Time & Receipt SCOPED	MED	LOW	P2	Scoped to inventory needs: DB-backed adapter, lead time field, goods receipt. PO/Portal deferred to Procurement module.
FR-011	Cycle Counting	MED	MED	P2	Eliminates annual physical inventory disruption. Continuous accuracy improvement.
FR-015	API Docs & Webhooks INFRA	MED	LOW	P1	Low effort (annotation-based). Cross-cutting infrastructure. Platform-level concerns (event bus, ERP feeds) deferred.
FR-003	Barcode Scanning	MED	MED	P3	Significant UX improvement for warehouse floor. Requires mobile support (FR-010).
FR-004	Multi-Warehouse	HIGH	HIGH	P3	Major capability gap. Complex workflow and data model changes.

FR-010	Mobile PWA	MED	MED	P3	Leverages existing React codebase. Maximum value when combined with FR-003.
FR-012	Lot Tracking	MED	HIGH	P3	Industry-specific need (pharma, food, chemicals). Required for FR-014 FIFO/LIFO.
FR-013	Costing Methods	MED	MED	P3	Financial reporting accuracy. Depends on FR-012 lot data for FIFO/LIFO.

7. Technical Architecture Evolution

7.1 Current Architecture (Post-Migration)

```
┌──────────────────────────────────────────────────────────────┐
│                      React 18 Frontend                       │
│  SCR-001 Menu │ SCR-002 List │ SCR-003 Detail │ SCR-006 Rpt │
└──────────────────────────────┬───────────────────────────────┘
                               │ REST API (JSON)
┌──────────────────────────────┴───────────────────────────────┐
│                    Spring Boot 3.3 Backend                   │
│                                                              │
│  ┌────────────┐   ┌────────────┐   ┌──────────────────┐     │
│  │ Controllers │→│  Services   │→│   Repositories    │     │
│  └────────────┘   └────────────┘   └──────────────────┘     │
│        ↑               ↑                    │               │
│  ┌─────┴──────┐   ┌─────┴──────┐            │               │
│  │   DTOs     │   │ Validators │            │               │
│  └────────────┘   └────────────┘            ↓               │
│  ┌──────────────────────────────────────────────────┐       │
│  │  Adapters (Static Maps): Category, Supplier, WH  │       │
│  └──────────────────────────────────────────────────┘       │
└──────────────────────────────┬───────────────────────────────┘
                               │ JPA / Hibernate
┌──────────────────────────────┴───────────────────────────────┐
│                PostgreSQL 16 — trtct004                      │
│   itmmst │ ctgmst │ suplmst │ whsmst │ invtrn (dormant)     │
└──────────────────────────────────────────────────────────────┘
```

7.2 Target Architecture (Post-Reimagination)

```
┌──────────────────────────────────────────────────────────────────┐
│                      React 18 Frontend + PWA                     │
│  Dashboard│List│Detail│Reorder│Receipt│CycleCount│History│Mobile │
│  ┌────────────┐   ┌──────────┐   ┌──────────────┐               │
│  │ Barcode/QR │   │ Offline  │   │ Notifications │               │
│  └────────────┘   └──────────┘   └──────────────┘               │
└──────────────────────────────┬───────────────────────────────────┘
                               │ REST API (JSON) + WebSocket + SSE
┌──────────────────────────────┴───────────────────────────────────┐
│                      Spring Boot 3.3 Backend                     │
│                                                                  │
│  ┌──────────┐   ┌──────────────┐   ┌────────────┐   ┌──────────┐ │
│  │Controllers│→│   Services    │→│ Repositories │   │ Scheduled │ │
│  └──────────┘   └──────────────┘   └────────────┘   │   Jobs    │ │
```

```
│       ↑          ┌──────┴──────┐        │            └──────────┘ │
│   ┌───┴───┐      │ New Services│        │                         │
│   │  DTOs  │      │• Forecast   │        ↓                         │
│   │  +v2   │      │• ABC Calc   │  ┌──────────────┐               │
│   └───────┘      │• Audit Log  │  │  Event Bus   │               │
│                  │• Cycle Count│  │  (Spring)    │               │
│   ┌──────────┐   │• Receipt    │  └──────┬───────┘              │
│   │Validators│   │• Alerts     │         │                       │
│   └──────────┘   │• Transfer   │         ↓                       │
│                  └─────────────┘  ┌──────────────┐               │
│   ┌──────────────────────────┐    │  Webhooks &  │               │
│   │ Adapters (DB-Backed):    │    │  Notifier    │               │
│   │ Category, Supplier, WH   │    └──────────────┘               │
│   │ + Outbound: Email, Webhook │                                  │
│   └──────────────────────────┘                                    │
│                                                                    │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │ Spring Security (JWT + RBAC) — All endpoints secured      │   │
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────┬──────────────────────────────────────┘
                           │ JPA / Hibernate + Flyway V6-V12
┌──────────────────────────┴──────────────────────────────────────┐
│                    PostgreSQL 16 — trtct004                      │
│  itmmst (+ abcClass, costMethod) │ ctgmst │ suplmst (+lead_time) │
│  whsmst │ invtrn (ACTIVE!) │ lotmst │ serialmst                 │
│  whtransfer │ cyclecount │ count_schedule │ cost_history         │
│  notifications │ webhook_subs │ api_keys │ users │ roles         │
└──────────────────────────────────────────────────────────────────┘
```

7.3 Database Schema Evolution

MIGRATION	PHASE	CHANGES
V6	P1	Add `users`, `roles`, `user_roles` tables. Add `api_keys` table.
V7	P1	No schema change — activate INVTRN by wiring service writers.
V8	P2	Add `abcClass` CHAR(1) to itmmst. Add `notifications`, `notification_prefs` tables.
V9	P2	Extend suplmst with `lead_time_days` INTEGER. Add `cyclecount`, `count_schedule` tables.
V10	P3	Add `whtransfer` table. Add `lotmst`, `serialmst` tables.
V11	P3	Add `cost_method` to itmmst. Add `cost_history` table. Add `webhook_subs` table.
V12	P3	Performance indexes for new tables. Materialized views for dashboard aggregates.

8. Risk Assessment & Mitigation

#	RISK	LIKELIHOOD	IMPACT	MITIGATION STRATEGY
R1	**Regression of existing BRs**	MEDIUM	HIGH	Existing 120 unit tests form the regression suite. Run on every build. Add integration tests per phase. No BR may be modified without explicit approval.
R2	**Scope creep across phases**	HIGH	MEDIUM	Strict phase gates. Each FR has defined acceptance criteria. Defer-to-next-phase is the default for overruns.
R3	**Data migration complexity**	MEDIUM	MEDIUM	All schema changes via Flyway (V6–V12). Additive-only changes (new columns have defaults). No destructive schema operations.
R4	**Performance degradation from audit logging**	LOW	MEDIUM	Async event listeners for INVTRN writes. Separate write path from main transaction. Batch insert optimization for bulk operations.
R5	**Forecast accuracy insufficient for auto-reorder**	MEDIUM	LOW	Graceful degradation to ALG-002 static reorder. Forecast accuracy tracked and displayed. Auto-reorder requires human approval.
R6	**Mobile offline sync conflicts**	MEDIUM	MEDIUM	Last-write-wins for quantity adjustments with conflict notification. Conflict resolution screen for manual intervention.
R7	**Security vulnerabilities in new endpoints**	MEDIUM	HIGH	FR-013 (RBAC) is Phase 1 priority. All new endpoints secured before any external exposure. OWASP dependency check in CI pipeline.
R8	**User adoption resistance**	MEDIUM	MEDIUM	Phased rollout allows gradual adoption. Dashboard (FR-001) provides immediate visible value. Training sessions per phase.

9. Success Metrics & KPIs

#	METRIC	CURRENT BASELINE	PHASE 1 TARGET	PHASE 3 TARGET
M1	**Inventory Accuracy**	Unknown (no counting process)	Baseline established	≥98% (cycle counting)
M2	**Stockout Frequency**	Detected reactively only	Real-time visibility	50% reduction via forecasting

M3	**Reorder Lead Time**	Days (manual report → fax → wait)	Same-day visibility	<4 hours (auto-PO creation)
M4	**Item Maintenance Time**	~5 min per item (one-at-a-time)	<1 min (bulk operations)	<30 sec (barcode scan)
M5	**Audit Compliance**	0% (no audit trail)	100% (all CRUD logged)	100% + lot traceability
M6	**Report Generation**	Manual screen-by-screen only	On-demand export	Real-time dashboard + PDF
M7	**Screens / Capabilities**	6 screens / ~70 BRs	10 screens / ~110 BRs	14+ screens / ~180+ BRs
M8	**Test Coverage**	120 unit tests	250+ tests (unit + integration)	500+ tests (unit + integration + E2E)

10. Summary & Recommendation

10.1 What This Proposal Delivers

This proposal transforms the TRTCT004 Inventory Management System from a faithfully migrated legacy application into a **modern inventory intelligence platform** through **12 inventory-domain reimaginations** plus **2 cross-cutting infrastructure prerequisites** across 3 phases — all strictly scoped to inventory management:

Phase 1 — Foundation (Weeks 1–10)

- Activate dormant audit trail (INVTRN)
- Secure all endpoints with RBAC
- Replace static menu with intelligence dashboard
- Enable bulk operations and data export
- Publish API documentation

Phase 2 — Intelligence (Weeks 11–22)

- Predictive demand forecasting and EOQ
- ABC inventory classification
- Automated notification engine
- Supplier lead time awareness and goods receipt
- Cycle counting workflows

Phase 3 — Advanced (Weeks 23–36)

- Barcode/QR scanning integration
- Multi-warehouse transfer management
- Mobile PWA for warehouse floor operations
- Lot tracking, expiry, and serial number management
- Inventory valuation and costing methods

10.2 Recommendation

Recommended Approach: Begin with **Phase 1** immediately. The five Phase 1 items (3 inventory reimaginations + 2 infrastructure prerequisites) deliver the highest value with the lowest effort:

- **FR-005 (Audit Trail)** unlocks data for every subsequent feature — the single most important enabler.
- **FR-014 (RBAC)** INFRA is a production deployment prerequisite — no security means no production.
- **FR-001 (Dashboard)** delivers immediate visible value that builds stakeholder confidence.
- **FR-008 (Bulk Operations)** solves the #1 user pain point from the legacy system.
- **FR-015 (API Documentation)** INFRA is near-zero effort with springdoc-openapi annotations.

Phase 1 can be delivered in 8–10 weeks while preserving all 70+ existing business rules and 120 unit tests. Each subsequent phase decision should be informed by usage data and business priorities gathered during the preceding phase.

10.3 Legacy to Reimagined — The Complete Transformation

CAPABILITY	AS/400 LEGACY	CURRENT (MIGRATED)	REIMAGINED (PROPOSED)
Data Entry	Keyboard-only, 5250 terminal	Web forms, mouse+keyboard	Barcode scan, bulk import, auto-feed
Visibility	Run a report to find out	Filter and browse lists	Real-time dashboard with KPIs
Reorder	Print report → fax to supplier	View report on screen	Predicted demand → auto-PO draft
Audit	None (overwrite in place)	None (INVTRN table dormant)	Full event sourcing, every change traced
Security	OS-level user profiles	None (open endpoints)	JWT + RBAC + warehouse scoping
Analytics	None	Margin calculation only	ABC classification, forecasting, scorecards
Mobility	Desk-bound terminals only	Browser (responsive)	PWA with offline, camera, touch
Integration	Data queues, CALL/PARM	REST API (internal only)	OpenAPI, Webhooks, event bus
Suppliers	6-char code lookup	Static map adapter	DB-backed adapter, lead time, goods receipt
Counting	Annual physical (offline)	No counting capability	Continuous cycle counting (ABC-driven)